



Центр образовательных программ ФРКТ

Р. Л. Пальян, А. Г. Лапушкин, В. К. Манукян

ИССЛЕДОВАНИЕ АЛГОРИТМОВ И МЕТОДОВ ОПТИМИЗАЦИИ ВЫЧИСЛЕНИЙ В ЗАДАЧЕ МАТРИЧНОГО УМНОЖЕНИЯ

Учебно-методическое пособие
по курсу

Введение в распараллеливание алгоритмов и программ

МОСКВА
МФТИ, ФИЗТЕХ
2025

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования «Московский физико-технический институт (национальный
исследовательский университет)»

Центр образовательных программ ФРКТ

Р. Л. Пальян, А. Г. Лапушкин, В. К. Манукян

ИССЛЕДОВАНИЕ АЛГОРИТМОВ И МЕТОДОВ ОПТИМИЗАЦИИ ВЫЧИСЛЕНИЙ В ЗАДАЧЕ МАТРИЧНОГО УМНОЖЕНИЯ

Учебно-методическое пособие

по курсу

Введение в распараллеливание алгоритмов и программ

МОСКВА
МФТИ, Физтех
2025

УДК 004.421
ББК 32.973.05
П14

Рецензент
Кандидат технических наук *А. Л. Лилеин*

Пальян, Рубен Левонович,
Лапушкин, Андрей Георгиевич,
Манукян, Ваграм Каренович

**П14 Исследование алгоритмов и методов оптимизации вычислений
в задаче матричного умножения : учебно-методическое пособие**
по курсу Введение в распараллеливание алгоритмов и программ /
Р. Л. Пальян, А. Г. Лапушкин, В. К. Манукян. – Москва : МФТИ, Физтех,
2025. – 24 с.

Рассматривается проблема оптимизации алгоритмов умножения матриц, как со стороны применения алгоритмов быстрого умножения матриц, так и со стороны оптимизации реализации под современные архитектуры вычислительных систем, включая распараллеливание и векторизацию.

Работа предназначена для студентов 4 курса, изучающих курс *Введение в распараллеливание алгоритмов и программ*.

УДК 004.421
ББК 32.973.05

Учебное издание
Пальян Рубен Левонович, Лапушкин Андрей Георгиевич,
Манукян Ваграм Каренович

ИССЛЕДОВАНИЕ АЛГОРИТМОВ И МЕТОДОВ ОПТИМИЗАЦИИ ВЫЧИСЛЕНИЙ В ЗАДАЧЕ МАТРИЧНОГО УМНОЖЕНИЯ

Учебно-методическое пособие по курсу
Введение в распараллеливание алгоритмов и программ

Редактор *Дружинина В.А.* Корректор *Волкова И.А.*

Подписано в печать 07.05.2025. Формат 60 × 84 ¹/₁₆. Усл. печ. л. 1,5.

Уч.-изд. л. 1,0. Тираж 20 экз. Заказ № 94.

Федеральное государственное автономное образовательное учреждение
высшего образования «Московский физико-технический институт
(национальный исследовательский университет)»

141700, Московская обл., г. Долгопрудный, Институтский пер., 9

Тел. (495) 408-58-22, e-mail: rio@mipt.ru

Отдел оперативной полиграфии «Физтех-полиграф». 141700, Московская обл.,

г. Долгопрудный, Институтский пер., 9. Тел. (495) 408 84 30, e-mail: polygraph@mipt.ru

© Пальян Р. Л., Лапушкин А.Г., Манукян В.К., 2025
© Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)», 2025

СОДЕРЖАНИЕ

Введение	4
Алгоритмы умножения матриц и их сложность	5
Кэш память современных процессоров и оптимизация перемножения матриц.....	8
Инструмент для создания параллельных программ для систем с общей памятью – OpenMP API	9
SIMD расширения систем команд.....	10
Методика исследования производительности	15
Задание к допуску	17
Задание к выполнению	17
Задание к сдаче.....	18
Контрольные вопросы	19
Приложение А. Образец программы перемножения квадратных матриц малого и среднего размера на языке программирования Си.....	19
Список сокращений.....	24
Список литературы.....	24

Введение

Среди базовых алгоритмов, широко используемых в настоящее время, алгоритмы умножения матриц занимают особое место.

С одной стороны, сложно переоценить практическую значимость данных алгоритмов. Можно упомянуть уже ставшие классическими задачи, такие как целый ряд задач вычислительной математики. Зачастую они сводятся к алгоритму умножения матриц (как к требующему больше всего вычислений из используемых алгоритмов), или, по крайней мере, именно этот алгоритм для своего выполнения потребляет существенную часть вычислительных ресурсов, необходимых для решения таких задач. Современной тенденцией является массовое использование алгоритмов машинного обучения, основанных на таких технологиях, как, например, свёрточная нейронная сеть (convolutional neural network, CNN). Многие реализации методик машинного обучения включают в себя алгоритмы умножения матриц.

С другой стороны, имеющиеся алгоритмы умножения матриц имеют относительно высокую асимптотическую сложность вычислений. Речь идёт о сложности $O(n^3)$ для базового алгоритма, несколько меньше для алгоритмов быстрого умножения матриц, поиск которых продолжается и в нынешние дни (например, в качестве одной из последних работ можно упомянуть (1)), а сама задача пока что считается одной из нерешённых проблем линейной алгебры.

При решении практических задач, кроме выбора алгоритма по его асимптотической сложности, часто встают вопросы прикладного характера по организации вычислений таким образом, чтобы добиться оптимального использования имеющихся аппаратных ресурсов. В частности, часто встречаются: оптимизация работы с памятью (с целью минимизации промахов в кеш-память), использование параллельных и массово-параллельных систем (в том числе специализированных ускорителей, например GPGPU), SIMD-расширений систем команд процессоров общего назначения, ускорителей на базе FPGA и др. Стоит отметить и появление специализированных аппаратных модулей, которые могут играть, в том числе роль матричных умножителей, таких как тензорные ядра в GPGPU, специализированных NPU и др.

Цель данной работы – демонстрация при помощи примера умножения квадратных матриц с данных приёмов. В частности, студентам предлагается ознакомиться с базовыми принципами оптимизации работы с памятью, использованием параллелизма, SIMD-расширений систем команд и использования вычислительных возможностей GPGPU.

Алгоритмы умножения матриц и их сложность

Для совместимых матриц A и B (т.е., когда число столбцов матрицы A совпадает с числом строк матрицы B) размерами M на K и K на N соответственно, результатом будем считать матрицу C размером M на N элементов, вычисляемых по формуле:

$$C[i, j] = \sum (A[i, k] * B[k, j]); \quad (1)$$

Для упрощения, в данной работе будем рассматривать частный случай, когда все матрицы (A , B и C соответственно) квадратные. Также матрицы, являющиеся операндами, считаются полными – т.е. большинство элементов ненулевые. Оптимизация процесса перемножения разреженных матриц несомненно представляет большой интерес в целом классе задач, однако выходит за рамки данной работы.

При такой постановке задачи асимптотическая вычислительная сложность базового, т.е. основанного на определении алгоритма умножения матриц, будет $O(n^3)$, где n – размер перемножаемых матриц. При этом программный код такой операции на языке C может выглядеть таким образом:

```
void matrix_mul_naive( int N, const double * A, const double * B, double * C)
{
    for (int i = 0; i < N; ++i)
    {
        for (int j = 0; j < N; ++j)
        {
            C[i*N + j] = 0;
            for (int k = 0; k < N; ++k)
                C[i*N + j] += A[i*N + k] * B[k*N + j];
        }
    }
}
```

Кроме вычислительной сложности, в данном фрагменте кода проявляется ряд проблем, связанных с архитектурой современных вычислительных систем. В частности с особенностями их работы с оперативной памятью. Так, для матрицы B идут последовательные обращения по адресам, удалённым на расстояние $N * \text{sizeof}(\text{double})$ в памяти. Это, при достаточно больших N , будет приводить к постоянным промахам в кеш-памяти, а значит и к серьёзному понижению скорости работы программы.

Соответственно в зависимости от размера матриц выявляются следующие пути ускорения процесса матричного перемножения: поиск алгоритма с асимптотической сложностью меньшей, чем $O(n^3)$ (что особенно актуально для матриц большого размера), оптимизация

программы под существующие архитектуры вычислительных систем (в частности организация оптимальной работы подсистем памяти, а также использования всех имеющихся аппаратных ресурсов, что подразумевает распараллеливание в MIMD- и SIMD-моделях параллелизма или в их комбинации. В данной работе MIMD-модель будет представлена на примере OpenMP API, SIMD – на примере SIMD расширений системы команд в архитектуре x86. Стоит отметить, что на практике активно применяются также, при наличии таковых, ускорители вычислений, например, GPGPU, FPGA и т.д. (не рассматриваются в данной работе).

Вернёмся к алгоритмам перемножения относительно больших матриц. Первый приём, который часто применяется на практике, это сведение процесса перемножения больших матриц к процессу перемножения средних и небольших матриц, что, в частности, позволяет проводить такие операции в пределах объёма кэш-памяти процессора или использовать имеющиеся аппаратные ускорители. Данный метод, именуемый *блочным умножением матриц*, может сочетаться также со всеми видами параллелизма и, хотя и не изменяет общее число операций, позволяет выполнить исходное умножение быстрее за счёт вышеуказанных оптимизаций под современные вычислительные архитектуры. Предположим, пусть рассматривается умножение двух матриц, A и B, с результатом $C = AB$. Если при этом итоговую матрицу C можно разбить на $p \times s$ блоков, то, вместо формального определения (1) результат можно представить в виде

$$C = AB = \begin{pmatrix} \sum_{i=1}^q A_{1i} B_{i1} & \sum_{i=1}^q A_{1i} B_{i2} \cdots & \sum_{i=1}^q A_{1i} B_{is} \\ \sum_{i=1}^q A_{2i} B_{i1} & \sum_{i=1}^q A_{2i} B_{i2} \cdots & \sum_{i=1}^q A_{2i} B_{is} \\ \dots & \dots & \dots \\ \sum_{i=1}^q A_{pi} B_{i1} & \sum_{i=1}^q A_{pi} B_{i2} \cdots & \sum_{i=1}^q A_{pi} B_{is} \end{pmatrix} \quad (2)$$

Представление в виде (2) позволяет как выполнить задачу распараллеливания вычислений, так и произвести оптимизацию работы с кэш-памятью (в идеальной ситуации до такого размера матриц, которые умещаются в L1 кэш-памяти процессора). К тому же этим методом мы можем свести умножение больших матриц к множественной операции умножения средних или малых матриц, для чего вполне можно применить как SIMD-расширения системы команд, так и другие аппаратные возможности (например, GPU). Блочный алгоритм также встречается под обозначением «разделяй и властвуй (divide-and-conquer)».

Особенно просто и лаконично данный метод смотрится для ситуации умножения квадратных матриц размером 2^k , где k – целое положительное число.

На основе блочного алгоритма создан и распространённый

алгоритм с асимптотической сложностью меньше, чем $O(n^3)$ – алгоритм Штрассена (автор – Фолькер Штрассен). На данный момент этот алгоритм не является таковым с самой низкой степенью n по вычислительной сложности, однако обладает относительно простой реализацией. К тому же некоторые из рекордных по этому параметру алгоритмов обладают большой константной частью сложности, из-за чего практическая выгода по времени исполнения от их применения проявляется только на матрицах очень большого размера. Благодаря этому данный алгоритм и сейчас применяется при решении практических задач.

Пусть A и B – матрицы размера 2^k . Их можно представить как блочные матрицы размера 2×2 , состоящие из $(2^{k-1} \times 2^{k-1})$ матриц.

Представим матрицу A как $\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$, а матрицу B как $\begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$.

В таком случае, результат произведения – матрицу C , можно представить, согласно формуле блочного умножения, в виде

$$\begin{pmatrix} A_{11}B_{11} + A_{12}B_{21} & A_{11}B_{12} + A_{12}B_{22} \\ A_{21}B_{11} + A_{22}B_{21} & A_{21}B_{12} + A_{22}B_{22} \end{pmatrix} \quad (3)$$

Можно заметить, что в правой части имеется восемь умножений матриц размера $(2^{k-1} \times 2^{k-1})$.

Пусть:

$$\begin{aligned} D &= (A_{11} + A_{22})(B_{11} + B_{22}); \\ D_1 &= (A_{11} + A_{22})(B_{11} - B_{22}); \\ D_2 &= (A_{21} - A_{11})(B_{11} + B_{12}); \\ H_1 &= (A_{11} + A_{12})B_{22}; \\ H_2 &= (A_{21} + A_{22})B_{11}; \\ V_1 &= A_{22}(B_{21} - B_{11}); \\ V_2 &= A_{11}(B_{11} - B_{22}); \end{aligned}$$

Тогда можно представить матрицу C как:

$$C = \begin{pmatrix} D & 0 \\ 0 & D \end{pmatrix} + \begin{pmatrix} D_1 & 0 \\ 0 & D_2 \end{pmatrix} + \begin{pmatrix} -H_1 & H_1 \\ H_2 & -H_2 \end{pmatrix} + \begin{pmatrix} V_1 & V_2 \\ V_1 & V_2 \end{pmatrix}, \text{ или же}$$

$$C = \begin{pmatrix} D+D_1+V_1-H_1 & V_2+H_1 \\ V_1+H_2 & D+D_2+V_2-H_2 \end{pmatrix} \quad (4)$$

Таким образом, мы избавляемся от одной из операций умножения матриц. Данный алгоритм можно использовать рекурсивно до достижения тривиальных операций, для каждого из оставшихся семи матричных умножений. Вычислительная сложность алгоритма при рекурсивной

реализации будет $O(n^{\log_2 7})$.

На практике, данный метод, являясь родственным блочному алгоритму умножения матриц, может применяться совместно с различными видами распараллеливания, а также совместно с аппаратным ускорением умножения матриц малого и среднего размера. Для этого достаточно в какой-то момент вместо продолжения рекурсии, использовать соответствующие способы умножения матриц.

Кэш-память современных процессоров и оптимизация перемножения матриц

Современные процессоры имеют многоуровневую кэш память (L1, L2, L3), которая работает значительно быстрее основной оперативной памяти. При этом данные в кэш-память загружаются автоматически при выполнении операций с основной памятью, а управлять процессом со стороны прикладной программы можно только косвенно, организуя работу с данными с учётом организации кэш-памяти. Кэш-память состоит из строк кэша (cache lines), обычно длина одной строки на текущий момент составляет 64 байта, но в общем случае может варьироваться в зависимости от архитектуры процессора. Когда процессор запрашивает данные, то проверяется наличие в кэше. При отсутствии возникает ситуация промаха в кэше (cache miss), в таком случае загружается вся строка из более медленной памяти. В то время, когда данные загружаются, процессор для данной задачи фактически простаивает. Так как процессор оперирует линиями кэша, то при запросе даже одного байта в кэш сохранится вся линия в 64 байта, из-за этого при «наивном» перемножении матриц (которое проводится по формуле – определению матричного перемножения) возникает такое явление, как «замусоривание кэша» (cache thrashing). Это происходит из-за того, что от первого операнда требуется строка — последовательно лежащие ячейки памяти, загрузка которых благотворно сказывается на оптимальности перемножения, а от второго операнда требуется столбец. При разыменовании столбца как раз и происходит замусоривание кэша, так как нам нужно, например, значение одной переменной типа float, для чего требуется прочитать из оперативной памяти 4 байта. Проблема состоит в том, что при загрузке этого значения в кэш поступает сразу 64 байта, т.е. искомые 4 байта с соседствующими данными в количестве 60 байт. Из этой ситуации можно найти несколько выходов. Эффективным способом является блочное перемножение, при котором следует выбрать блок с таким размером, при котором три блока (операнды и результат) будут полностью помещаться в кэш процессора. Простым, но всё равно дающим неплохие результаты в сравнении с «наивным» алгоритмом, является вариант, в котором требуется

транспонировать второй операнд, с тем, чтобы перемножать строку на строку — два последовательно расположенных массива данных. Первый способ хорош для перемножения больших матриц, но немного сложнее в реализации, второй способ хорошо подходит для перемножения малых и средних матриц на центральном процессоре и прост в реализации.

Инструмент для создания параллельных программ для систем с общей памятью – OpenMP API

Для создания прикладных параллельных программ, предназначенных для систем с общей памятью, может использоваться API, соответствующий стандарту OpenMP. Данный стандарт имеет ряд версий (последняя по данным на начало 2025 года – 6.0), и развивается организацией OpenMP Architecture Review Board, в которую входят ряд коммерческих и научных организаций, заинтересованных в развитии и активном использовании стандарта, преимущественно для решения прикладных задач науки и инженерии. Различные версии стандарта поддерживаются большинством современных компиляторов языков C, C++ и Fortran общего назначения.

Основой стандарта являются: набор директив компилятора, библиотечные функции и переменные окружения.

Для использования API OpenMP в языке C при использовании компилятора gcc необходимо при компиляции исходного кода программы дополнительно задать опцию компилятора *-fopenmp*. В тексте самой программы требуется также подключить библиотеку *omp.h*.

Для параллельного выполнения блока кода предназначена директива *omp parallel*. Ниже приведён пример использования данной директивы:

```
#pragma omp parallel
{
    //код, который будет выполнен параллельно
}
```

При этом системой будет создано несколько потоков (число зависит от значения соответствующей глобальной переменной или ранее установленного в программе значения), которые будут выполняться одновременно, имея как локальные, так и общие для всех области памяти. При работе с последними, чтобы избежать некорректных результатов вычислений из-за так называемых «гонок», т.е. ситуаций, когда несогласованная работа разных потоков с одними и теми же данными

приводит к получению некорректного результата, можно использовать встроенные в API элементы синхронизации (например, `omp_lock`).

Для распараллеливания циклов по итерациям имеется отдельная директива `for`, которая должна использоваться внутри параллельной области (допускается одновременное создание параллельной области вместе с использованием директивы `for`

```
#pragma omp parallel for
for(i=0; i < N;i++)
{
    //код, который будет выполняться параллельно по итерациям
цикла
}
```

При этом цикл должен соответствовать ряду требований, которые отличаются в зависимости от версии стандарта. Основные требования: определённое заранее число итераций и отсутствие дополнительных точек выхода из цикла. Стоит отметить, что система не занимается выявлением информационных зависимостей между итерациями циклов, корректность применения такого вида распараллеливания должна контролироваться программистом.

Стандарт OpenMP содержит встроенный механизм измерения времени работы фрагмента кода при помощи функции `omp_get_wtime()`. С помощью данной функции можно получить астрономическое время в секундах (вещественное число), прошедшее с некоторого момента в прошлом. Ниже приведено типовое использование данной функции:

```
double t_start = omp_get_wtime();

// код, время исполнения которого необходимо измерить

double t_end = omp_get_wtime();

double working_time = t_end - t_start;
```

SIMD-расширения систем команд

SIMD-расширения систем команд (Single-Instruction, Multiple-Data — "Одна инструкция, множество данных") — это набор инструкций, предназначенных для параллельной обработки данных без использования многопоточности, который позволяет одновременно выполнять одну операцию над несколькими элементами данных, что значительно

повышает производительность программ, особенно в задачах, связанных с обработкой больших объемов подряд расположенных данных. Под класс задач, ускоряемых с помощью SIMD, попадают задачи, связанные с мультимедиа, научные вычисления, машинное обучение и другие.

Такие расширения есть в большинстве современных архитектур общего назначения.

Рассмотрим работу данного механизма на примере архитектуры x86-64, где SIMD-инструкции реализованы через несколько поколений расширений: MMX, SSE, SSE2, SSE3, SSE4, SSE4.1, SSE4.2, AVX, AVX2, AVX512.

Первой реализацией SIMD был набор инструкций MMX или Multi Media Extension (мультимедийное расширение). MMX впервые появился в процессорах Pentium MMX в 1997 году. Для расширения были внедрены дополнительные регистры размером 64 бита (при этом архитектура самого процессора была 32-битная), с названиями `mm0` — `mm7`. Одна MMX-инструкция при работе с данными регистрами может применяться к двум 32-битным целым числам, четырем 16-битным целым числам или восьми 8-битным целым числам одновременно. MMX поддерживал только целочисленные вычисления. Одной из попыток развития технологии за счёт поддержки реализации работы с числами с плавающей точкой с использованием векторных регистров было представленное компанией AMD расширение 3DNow!, которую можно считать боковой веткой развития технологии. Затем набор инструкций MMX был дополнен набором инструкций Streaming SIMD Extension (SSE).

SSE — набор инструкций, разработанный компанией Intel и впервые представленный в процессорах серии Pentium III. Например, процессор, поддерживающий SSE, имеет 8 (в первой версии, в дальнейшем 16) дополнительных 128-битовых регистров (от `xmm0` до `xmm7` или `xmm15`, в зависимости от версии) и управляющий регистр `mxcsr`. Регистры `xmm` могут содержать скалярные данные (*scalar data*) или упакованные данные (*packed data*). Под скалярными данными подразумевается только одно значение, занимающее целиком регистр. Если число помещается в регистр `xmm0`, то `xmm0` содержит скалярное значение. Но в `xmm0` также можно записать несколько значений, тогда эти значения определяются как упакованные данные. Возможные варианты сохранения значений в любом `xmm`-регистре:

- два 64-битовых числа с плавающей точкой двойной точности;
- четыре 32-битовых числа с плавающей точкой обычной (одинарной) точности;
- два 64-битовых целых числа (четверные слова);
- четыре 32-битовых целых числа (двойные слова);
- восемь 16-битовых коротких (*short*) целых чисел (слова);

- шестнадцать 8-битовых байт или символов.

Advanced Vector Extensions (AVX) – это следующее получившее широкое распространение расширение системы команд для процессоров x86_64 (не существует 32-битных процессоров с поддержкой AVX). Если последние версии SSE предлагали 16 xmm-регистров размером 128 бит каждый, то система AVX предоставляет 16 ymm-регистров размером 256 бит каждый. Нижняя (младшая) половина каждого ymm-регистра в действительности представляет собой соответствующий xmm-регистр.

AVX 512 – это дальнейшее развитие технологии, в данном случае программисту предоставляется 32 zmm-регистра размером 512 бит каждый.

В дополнение к ymm-регистрам система AVX расширяет инструкции SSE и предоставляет набор дополнительных новых инструкций.

Получить доступ к векторным инструкциям можно и в программе на языке высокого уровня (в частности, C/C++) без ассемблерных вставок. Для этого используются так называемые *интринсики* (intrinsics) — встроенные объекты компилятора. В заголовочном файле объявлен один или несколько типов данных (с точки зрения программиста, это массив фиксированной длины, но без возможности доступа к элементам этого массива), переменной одного из этих типов соответствует векторный регистр. Также в заголовочном файле объявлены функции, которые принимают аргументы, возвращают значения указанных выше типов и производят с точки зрения программиста те же операции над данными, что и соответствующие им векторные инструкции. В действительности же настоящей программной реализации этих функций не существует: компилятор при генерации объектного кода заменяет вызов функции векторной инструкцией. Таким образом, с помощью интринсиков можно написать программу на языке высокого уровня, близкую или равную по производительности программе, написанной на языке ассемблера. Для доступа к интринсикам достаточно подключить соответствующий заголовочный файл. Интринсики не являются частью стандартов языков C/C++, но поддерживаются самыми распространёнными компиляторами, в частности, GCC, Clang, MSVC, Intel. Интринсики позволяют также упорядочить работу с разнообразными типами обрабатываемых данных. Важно отметить, что процессору (по крайней мере, в архитектуре x86) недоступна информация о типе данных, содержащихся в регистре. При выполнении той или иной векторной инструкции они интерпретируются как данные определённого, связанного с инструкцией типа: числа с плавающей запятой, целого числа некоторой разрядности со знаком или без знака. Контролировать корректность производимых вычислений при этом должен программист, что требует немалого внимания. Тем более, иногда тип данных может изменяться: например, при целочисленном умножении

разрядность произведения равна сумме разрядностей множителей. Интринсики позволяют в некоторой степени упростить задачу.

Тип данных обычно называется `__mxxx (T)`, где `xxx` представляет количество битов данных, например `__m128`, предоставляемый SSE, равен 128 битам, а `__m256`, предоставляемый AVX, является 256 бит. `T` обозначает тип упакованных данных, если это тип с плавающей запятой одинарной точности, он опускается, если это целое число, это `i`, например `__m128i`, если это тип с плавающей точкой двойной точности, это `d`, например `__m256d`.

Так, векторным регистрам `xmm (SSE)` соответствуют три типа данных:

- `__m128`, «массив» из четырёх чисел с плавающей запятой с одинарной точностью;

- `__m128d`, «массив» из двух чисел с плавающей запятой двойной точности;

- `__m128i`, 128-битный регистр, который можно рассматривать как «массив» 8-, 16-, 32- и 64-битных чисел.

Поскольку конкретная векторная инструкция работает, как правило, с одним из трёх типов данных (число с плавающей запятой одинарной точности, двойной точности, целочисленный), то и аргументы функций-векторных инструкций имеют один из трёх указанных типов.

Инструкции SSE / AVX в основном определены в следующих файлах заголовков:

- `<xmmintrin.h>`: SSE, поддерживает одновременные операции с 4 32-битными числами с плавающей запятой одинарной точности.

- `<emmintrin.h>`: SSE 2, поддерживает одновременные операции с двумя 64-битными числами с плавающей запятой двойной точности.

- `<pmmintrin.h>`: SSE 3, поддерживает горизонтальные операции срегистрами SIMD, такими как `hadd`, `hsub` и т. д.

- `<tmmintrin.h>`: SSSE 3, добавлены дополнительные инструкции.

- `<smintrin.h>`: SSE 4.1, поддерживает умножение точек и другие операции формирования.

- `<nmmintrin.h>`: SSE 4.2, добавлены дополнительные инструкции. (Также поддерживаются возможности все предыдущих версий SSE)

- `<immintrin.h>`: AVX, поддерживает одновременную работу с 8 числами с плавающей запятой одинарной точности или 4 числами с плавающей запятой двойной точности.

Для GCC существует заголовочный файл `<x86intrin.h>`, который включает сразу все интринсики.

Интринсики именуются по следующим правилам: `__mm (xxx)_name_PT`, где `xxx` обозначает количество бит регистра SIMD, если 128 бит, то `xxx` опускается, например `__mm_addsub_ps`, если 256 бит, то 256,

например `_mm256_add_ps`, `name` – это имя операции, выполняемой функцией, например `_mm_add_ps` для сложения и `_mm_sub_ps` для вычитания. Встроенных функций достаточно много, однако на сайте Intel есть удобное веб-приложение, которое позволяет найти требуемые для тех или иных задач функции.

При решении задачи перемножения матриц оптимальным способом важным является случай быстрого перемножения умножения малых и средних матриц. К этому же случаю, благодаря использованию таких методов, как, например, блочное перемножение матриц, сводятся и ситуации, когда надо перемножить относительно большие матрицы, используя все доступные аппаратные ресурсы. В случае работы с большими матрицами целесообразно разделять работу на нити через механизмы операционной системы, сводя, по возможности, например тем же блочным методом, работу к умножению малых и средних матриц средствами одного процессорного ядра. И собственно для малых и средних матриц можно использовать параллелизм на уровне данных за счёт встроенных в процессор векторных инструкций, используя один поток команд. Целесообразность применения такого вида параллелизма в случае умножения малых и средних матриц может объясняться как с точки зрения деструктивного влияния на производительность таких явлений, как явление, называемое в англоязычной литературе «false sharing», при обработке малого количества памяти с нескольких ядер, так и временем, затраченным на системные вызовы инициализации или пробуждения другого/их потока/ов.

False sharing — это проблема снижения производительности, возникающая в многопоточных приложениях, когда несколько потоков одновременно обращаются к разным переменным, которые расположены в одной и той же кэш-линии (cache line) в памяти. Несмотря на то, что потоки работают с разными данными, из-за того, что эти данные находятся в одной кэш-линии, процессор вынужден синхронизировать кэши между ядрами, что приводит к значительным задержкам и снижению производительности.

В современных архитектурах наличие и использование SIMD-инструкций давно стало стандартной практикой при создании эффективных высокопроизводительных приложений, в том числе и при работе с матричными операциями. Как упоминалось выше, для x86 это расширения MMX, SSE и AVX. Для использования SIMD-инструкций требуется выравнивание данных, для MMX это выравнивание данных по 8 байт, SSE — 16 байт, AVX — 32 байта. В современных SIMD-расширениях присутствуют инструкции и для не выровненных данных, однако если данные не выровнены, процессору может потребоваться выполнить дополнительные операции для загрузки данных из памяти, что замедляет выполнение.

Методика исследования производительности

В данной работе для ознакомления с технологиями умножения матриц предлагается использовать любую систему многопоточного программирования (например, OpenMP) и SIMD интринсики с использованием языков Си или C++. В языке Си при использовании компилятора GCC интринсики подключаются заголовочными файлами `x86intrin.h`.

Исследование производительности при использовании современных многозадачных ОС и CPU с изменяемой тактовой частотой требует максимально возможного ограничения работы фоновых процессов, отключения режимов энергосбережения, а для ноутбуков, по возможности, подключения сетевого блока питания. Также рекомендуется проводить измерения несколько раз и выбирать среднее полученное значение.

При умножении средних матриц важно, чтобы данные обоих операндов были локализованы в подряд лежащих кеш-линиях, для обеспечения пространственной локальности. Для случая малых матриц влияние этого эффекта сводится на нет благодаря тому, что матрица за раз помещается в несколько кеш-линий.

В приложении А приводится код программы, в которой происходит перемножение квадратных матриц маленького и среднего размера, от 8 до 512 элементов. Перемножение осуществляется тремя способами.

Первый — это наивное перемножение двух матриц тремя циклами.

Второе — первая матрица умножается на вторую, предварительно транспонированную матрицу. При этом достигается оптимальность по обращениям в кеш.

Третье — это перемножение с оптимизацией для кеша через широкие SIMD регистры, в данном случае SSE.

Ниже приводится пример результатов и графика с исследованиями времени перемножений этими тремя способами:

Таблица 1. Время перемножения в зависимости от размера матриц, на процессоре Intel(R) Core(TM) i5-2415M CPU @ 2.30GHz, с дополнительными флагами компилятора: -O2 -mavx

	Простое	Кеш опт.	Кеш опт. + SSE	Кеш опт. + AVX
8	0.00000186	0.00000178	0.00000081	0.00000078
16	0.00000716	0.00000646	0.00000253	0.00000237
32	0.00005337	0.00004146	0.00001083	0.00000803
64	0.00027493	0.00029919	0.00007521	0.00004553
128	0.00235544	0.00237568	0.00059906	0.00042855
256	0.02459493	0.01816406	0.00477301	0.00359345
512	0.23030529	0.14304150	0.03686060	0.02989318

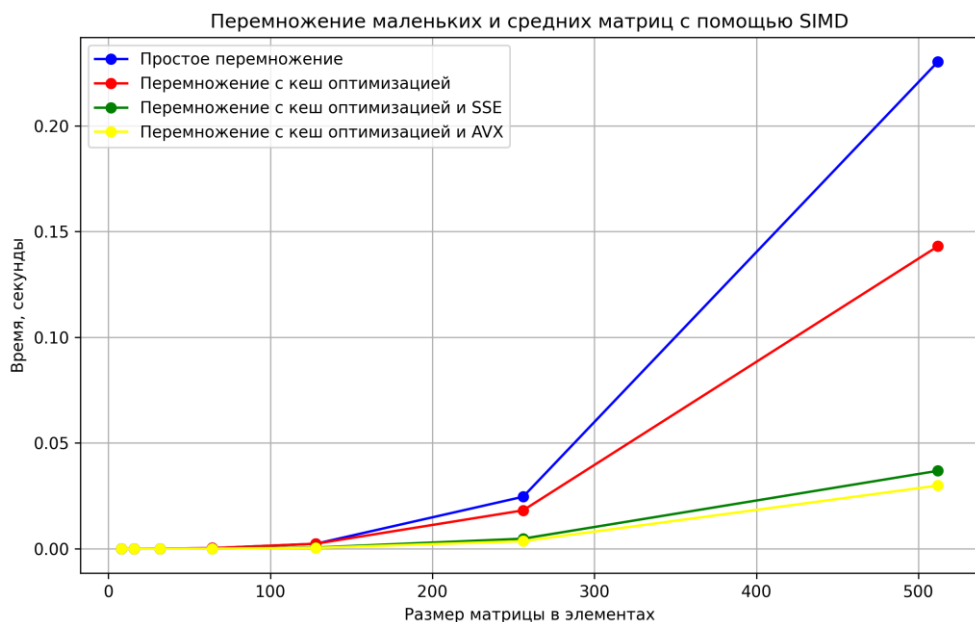


Рис. 1. Время перемножения на процессоре Intel(R) Core(TM) i5-2415M CPU @ 2.30GHz в зависимости от размера матриц

Таблица 2. Время перемножения в зависимости от размера матриц, на процессоре Intel(R) Core(TM) i9-9900K CPU @ 3.60GHz, с дополнительными флагами компилятора: -O2 -mavx

	Простое	Кеш опт.	Кеш опт. + SSE	Кеш опт. + AVX
8	0.00000153	0.00000124	0.00000091	0.00000101
16	0.00001175	0.00000776	0.00000286	0.00000070
32	0.00001646	0.00001177	0.00000416	0.00000345
64	0.00014730	0.00012299	0.00003232	0.00002084
128	0.00148058	0.00113670	0.00032741	0.00017664
256	0.01384467	0.01154296	0.00262659	0.00151103
512	0.17107172	0.10243039	0.02472121	0.01316139

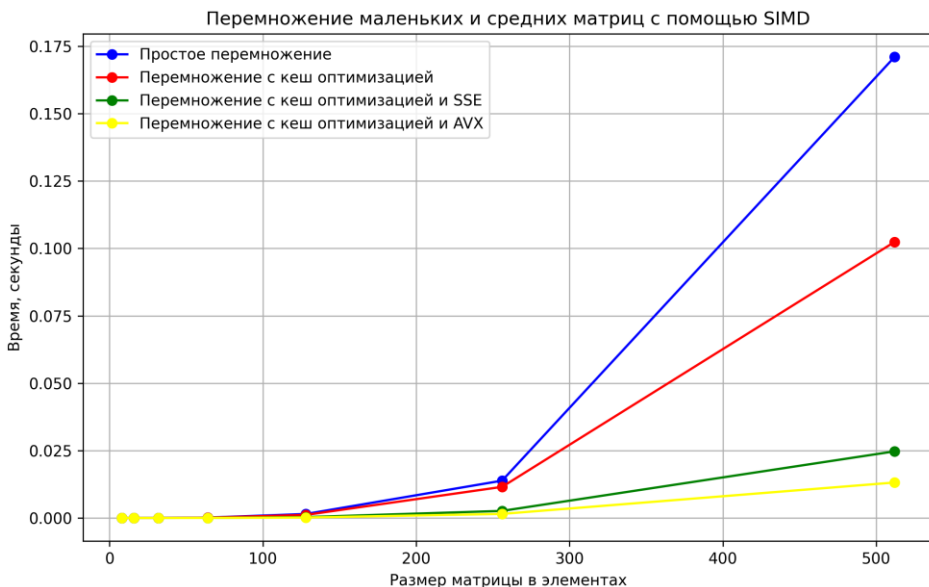


Рис. 1. Время перемножения на процессоре Intel(R) Core(TM) i9-9900K CPU @ 3.60GHz в зависимости от размера матриц

Задание к допуску

Создайте программу, которая осуществляет умножение двух квадратных матриц размера 512×512 базовым («наивным») алгоритмом. Скомпилируйте, при необходимости исправьте ошибки. Проверьте её работу на тривиальном случае, когда все поля исходной матрицы заполнены значением «1». Получите у преподавателя набор пунктов для выполнения и ряд исследуемых размеров матриц (например, 32, 64, 128, 256, 512, 1024, 2048 или другой, в зависимости от возможности используемого оборудования).

Задание к выполнению

1. Исследуйте производительность базовой («наивной») версии алгоритма умножения матриц. Для этого требуется провести измерение времени работы в зависимости от размера матрицы для выбранного ряда значений.
2. Проведите оптимизацию обращений к памяти, для чего транспонируйте одну из матриц для последовательного обращения к памяти.

- Скомпилируйте модифицированную версию программы, при необходимости исправьте ошибки. Удостоверьтесь в том, что результат работы программы не изменился. Проведите измерение времени работы в зависимости от размера матрицы для выбранного ряда значений.
3. Проведите оптимизацию обращений к памяти методом реализации блочного алгоритм. Скомпилируйте модифицированную версию программы, при необходимости исправьте ошибки. Удостоверьтесь в том, что результат работы программы не изменился. Проведите измерение времени работы в зависимости от размера матрицы для выбранного ряда значений.
 4. Реализуйте алгоритм быстрого умножения матриц (подразумевается алгоритм Штрассена, по желанию студента возможна реализация других существующих алгоритмов быстрого умножения матриц). Скомпилируйте соответствующую версию программы, при необходимости исправьте ошибки. Удостоверьтесь в том, что результат работы программы не изменился. Проведите измерение времени работы в зависимости от размера матрицы для выбранного ряда значений.
 5. Проведите распараллеливание одной из предыдущих версий перемножения матриц с использованием OpenMP или любого иного стандарта распараллеливания с использованием общей памяти. Скомпилируйте соответствующую версию программы, при необходимости исправьте ошибки. Удостоверьтесь в том, что результат работы программы не изменился. Проведите измерение времени работы в зависимости от размера матрицы для выбранного ряда значений.
 6. Проведите векторизацию перемножения матриц с использованием SIMD-инструкций CPU. Скомпилируйте соответствующую версию программы, при необходимости исправьте ошибки. Удостоверьтесь в том, что результат работы программы не изменился. Проведите измерение времени работы в зависимости от размера матрицы для выбранного ряда значений.

Задание к сдаче

1. Создайте программу, сочетающую в себе все проделанные в предыдущих пунктах оптимизации матричного перемножения. Скомпилируйте соответствующую версию программы, при необходимости исправьте ошибки. Удостоверьтесь в том, что результат работы программы не изменился. Добейтесь минимально возможного времени выполнения для выбранного размера. (При наличии условий возможно выполнение данного пункта в формате соревнования в пределах группы, размер обычно выбирается достаточно большой, т.е. 2048 или 4096 элементов).

2. Постройте графики зависимости времени от размера перемножаемых матриц для всех выполненных ранее пунктов.
3. Посчитайте ускорение и эффективность, в зависимости от размера перемножаемых матриц, для всех пунктов, где подразумевалось распараллеливание алгоритма. Постройте соответствующие графики.

Контрольные вопросы

1. Назовите асимптотическую вычислительную сложность базового алгоритма умножения матриц.
2. Что Вам известно про алгоритмы быстрого умножения матриц и их вычислительную сложность?
3. Что такое кэш-память и как механизмы её работы влияют на время работы алгоритмов умножения матриц?
4. Опишите блочный алгоритм умножения матриц.
5. Опишите алгоритм Штрассена умножения матриц.
6. Возможно ли распараллеливание алгоритмов умножения матриц?
7. Как можно использовать SIMD-расширения системы команд CPU для матричного перемножения?
8. Что такое ускорение и эффективность параллельных программ?

Приложение А. Образец программы перемножения квадратных матриц малого и среднего размера на языке программирования Си

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>
#include <x86intrin.h>

#define SIZE 32

#define ROWS SIZE
#define COLS SIZE

#define SSE_PS_LEN 4
#define AVX_PS_LEN 8

#define SSE_MEMORY_ALIGN 16
#define AVX_MEMORY_ALIGN 32
#define SIMD_MEMORY_ALIGN AVX_MEMORY_ALIGN
```

```

#define TIMING(a) \
do { \
    struct timeval start_time, end_time; \
    gettimeofday(&start_time, NULL); \
    a; \
    gettimeofday(&end_time, NULL); \
    double elapsed_time = (end_time.tv_sec - start_time.tv_sec) + \
(end_time.tv_usec - start_time.tv_usec) / 1000000.0; \
    printf("Время выполнения %s: %.8f сек \n", #a, elapsed_time); \
} while (0)

void multiplyMatrix(float *mat1, float *mat2, float *res) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            res[i * COLS + j] = 0;
            for (int k = 0; k < COLS; k++) {
                res[i * COLS + j] += mat1[i * COLS + k] * mat2[k * COLS + j];
            }
        }
    }
}

void multiplyMatrixOptimized(float *mat1, float *matT2, float *res) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            res[i * COLS + j] = 0;
            for (int k = 0; k < COLS; k++) {
                res[i * COLS + j] += mat1[i * COLS + k] * matT2[j * COLS + k];
            }
        }
    }
}

void multiplyMatrixOptimizedSSE(float *mat1, float *matT2, float *res) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            __m128 acc = _mm_set_ps1(0.0f);
            for (int k = 0; k < COLS; k += SSE_PS_LEN) {
                __m128 op0 = _mm_load_ps(mat1 + i * COLS + k);
                __m128 op1 = _mm_load_ps(matT2 + j * COLS + k);
                __m128 mul = _mm_mul_ps(op0, op1);
                acc = _mm_add_ps(mul, acc);
            }
        }
    }
}

```

```

        res[i * COLS + j] = *((float*)&acc + 0) + *((float*)&acc + 1) +
        *((float*)&acc + 2) + *((float*)&acc + 3);
    }
}

```

```

void multiplyMatrixOptimizedAVX(float *mat1, float *matT2, float *res) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            __m256 acc = _mm256_set1_ps(0.0f);
            for (int k = 0; k < COLS ; k += AVX_PS_LEN) {
                __m256 op0 = _mm256_load_ps(mat1 + i * COLS + k);
                __m256 op1 = _mm256_load_ps(matT2 + j * COLS + k);
                __m256 mul = _mm256_mul_ps(op0, op1);
                acc = _mm256_add_ps(mul, acc);
            }
            res[i * COLS + j] = *((float*)&acc + 0) + *((float*)&acc + 1) +
            *((float*)&acc + 2) + *((float*)&acc + 3) + *((float*)&acc + 4) + *((float*)&acc
            + 5) + *((float*)&acc + 6) + *((float*)&acc + 7);
        }
    }
}

```

```

void transposeMatrix(float *mat, float *transposedMat) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            transposedMat[j * ROWS + i] = mat[i * COLS + j];
        }
    }
}

```

```

void printMatrix(float *mat) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%f ", mat[i * COLS + j]);
        }
        printf("\n");
    }
}

```

```

void fillMatrix( float *matrix ){
    for(int i = 0 ; i < ROWS * COLS ; ++i ){

```

```

        matrix[i] = (float)(i % 100) / i;
    }
}

void fillValueMatrix( float *matrix, float value){
    for(int i = 0 ; i < ROWS * COLS ; ++i){
        matrix[i] = value;
    }
}

int cmpMatrix( float *mat1, float *mat2 ){
    for(int i = 0 ; i < ROWS * COLS ; ++i){
        // printf("%f %f i:%d\n", mat1[i], mat2[i], i);
        if(abs(mat1[i] - mat2[i]) > 1E-6f){
            return 1;
        }
    }
    return 0;
}

float main() {
    float *mat1 = aligned_alloc(SIMD_MEMORY_ALIGN, ROWS * COLS *
sizeof(float));
    float *mat2 = aligned_alloc(SIMD_MEMORY_ALIGN, ROWS * COLS *
sizeof(float));
    float *matT2 = aligned_alloc(SIMD_MEMORY_ALIGN, ROWS * COLS *
sizeof(float));

    float *resultSimple = malloc(ROWS * COLS * sizeof(float));
    float *resultOptimize = malloc(ROWS * COLS * sizeof(float));

    {
        fillMatrix(mat1);
        fillMatrix(mat2);
        fillValueMatrix(resultSimple, 0.0f);
        TIMING(multiplyMatrix(mat1, mat2, resultSimple));
    }

    {
        printf("\n");
        fillMatrix(mat1);
        fillMatrix(mat2);
    }
}

```

```

    fillValueMatrix(matT2, 0.0f);
    fillValueMatrix(resultOptimize, 0.0f);

    TIMING(transposeMatrix(mat1, matT2));
    TIMING(multiplyMatrixOptimized( mat1, matT2, resultOptimize));

    int rc = cmpMatrix(resultOptimize, resultSimple);
    printf("matrix matching: %d\n", rc);
}

{
    printf("\n");
    fillMatrix(mat1);
    fillMatrix(mat2);
    fillValueMatrix(matT2, 0.0f);
    fillValueMatrix(resultOptimize, 0.0f);

    TIMING(transposeMatrix(mat2, matT2));
    TIMING(multiplyMatrixOptimizedSSE(mat1, matT2, resultOptimize));

    int rc = cmpMatrix(resultOptimize, resultSimple);
    printf("matrix matching: %d\n", rc);
}

free(mat1);
free(mat2);
free(matT2);
free(resultSimple);
free(resultOptimize);

return 0;
}

```


Список сокращений

API – Application Programming Interface, интерфейс прикладного программирования.

CPU – Central Processor Unit, центральный процессор вычислительной системы.

GPGPU – General-purpose computing on graphics processing units, графические процессоры видеокарт, используемые для осуществления вычислений общего назначения.

FPGA – field-programmable gate array, программируемая логическая интегральная схема.

MIMD – multiple instruction, multiple data, принцип организации вычислений, когда вычисления выполняются с использованием, одновременно, некоторого количества потоков инструкций и обрабатываемых с их помощью данных.

MMX – Multi Media Extension, «мультимедийное» расширение системы команд в архитектуре x86.

NPU – Neural Processing Unit, специализированный процессор или сопроцессор, предназначенный для аппаратного ускорения работы алгоритмов искусственных нейронных сетей.

OpenMP – Open Multi-Processing, стандарт API для создания параллельных программ.

SIMD – single instruction, multiple data, принцип организации вычислений, когда одна инструкция применяется для некоторого количества данных, тем самым обеспечивая параллелизм вычислений на уровне данных.

SSE – Streaming SIMD Extension, SIMD расширение системы команд x86.

Список литературы

1. Ran Duan, Hongxun Wu, Renfei Zhou, Faster Matrix Multiplication via Asymmetric Hashing, 2023, 2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS), Electronic ISBN 979-8-3503-1894-4
2. Эффективное распараллеливание циклов / составители Р. Л. Пальян, Д. А. Гаврилов, А. В. Леус, А. В. Филимонов, Москва : МФТИ, 2022.
3. Алексеев В.Б. О билинейной сложности умножения матриц размеров $m \times 2$ и 2×2 // Чебышевский сб. Тула : Тульский гос. педагог. ун-т, 2015. Т. 16, № 4. С. 11–27.
4. Карпов В.Е., Лобанов А.И., Численные алгоритмы и программы. Введение в распараллеливание, Москва : Физматкнига, 2014, ISBN 978-5-89155-234-0.
5. Даниэль Куэсвюрм. Профессиональное программирование на ассемблере x64 с расширениями AVX, AVX2 и AVX-512. Москва, ДМК Пресс, 2021.
6. Официальная страница документации OpenMP. URL:
<https://www.openmp.org/specifications/>